

SPM: Types of Parallelism

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Types of parallelism
 - Data parallelism
 - Stream parallelism
 - Task parallelism
- Conceptual examples

Basic Performance Metrics (recap)

- **Key definitions for system performance**

- **Latency (L):** is the time from the moment an input (request) arrives until the output (response) is ready to be delivered.
 - In the queueing theory and networking contexts, Latency is also known as **Response Time**. It includes any waiting time plus the service time
- **Completion time (T_c):** is the total time an application takes to complete all tasks, from start to delivery of all results
 - $L = T_c$ if there is only one task to compute for the application
 - Often referred to as the **makespan**
- **Service time (T_s):** the time required by the system to process a task, excluding waiting time
 - At steady state and full utilization, the time between two consecutive task completions approximates the service time.
- **Throughput :** it is the number of tasks completed per unit of time
 - At steady state and full utilization, it is the inverse of the service time ($\frac{1}{T_s}$);
- **Inter-arrival time (T_a):** the average time between two consecutive task arrivals at the system

Data parallelism

- Data parallelism considers an input **collection** of data and produces an output collection of data
 - In general, input and output collection sizes may differ
- **The same operation (or kernel function) is applied to data elements (or blocks of data elements) in parallel on multiple Workers**
- The primary data parallel patterns are (input : output cardinality):
 - **Map** (N : N) - example: convert a list of temperatures from Celsius to Fahrenheit
 - **Reduce** (N : 1 or N : M with $M < N$) - example: sum N numbers, or counts per class (M classes)
 - **Stencil** (N : N) - example: 2D convolution
 - **Scan** (N : N) - example: cumulative sums of N numbers
- Objective: **reduce the completion time**

Example: map-based translation of a book

- Translating a book with n pages requires a time proportional to: $T = n \cdot T_{page}$
- Parallelization strategy (**map pattern**): split the dataset; compute independently on each partition; merge the partial results. More precisely:
 - Split the book in k parts, where k is the number of Workers (helpers), and assign each part to a Worker
 - Let's call T_{split} the time to split the book and assign a book partition to a Worker
 - Let each Worker to do the translation independently on the assigned part (n/k pages)
 - Gather the translated parts and merge them into a single coherent book
 - Let's call T_{merge} the time to gather the partial results and combine them
- $T_{par} = k \cdot (T_{split} + T_{merge}) + \frac{n}{k} \cdot T_{page}$
 - We pay an **overhead** for splitting and merging but we can compute in parallel over different partitions
- If n is large enough and if $T_{split}, T_{merge} \ll T_{page}$, the speedup is about k :

$$S(p) \approx \frac{n \cdot T_{page}}{\frac{n}{k} \cdot T_{page}} = k$$

Data streams

- Data stream is a key concept in many parallel applications
- A **data stream** is a sequence of data elements of the same type made available over time, often in real time
- Unlike traditional **batch processing**, where large datasets (collections) are available when the program starts, data streams are:
 - **Continuous**: Data arrives constantly (or the stream length is sufficiently long)
 - **Dynamic**: The *volume*, *velocity*, and *variety* of data may change over time
 - **Time-sensitive**: Processing must occur with minimal delay

Stream parallelism

- A **stream of tasks** refers to a continuous, **ordered** sequence of computational or operational units (called **tasks**) **of the same type** that are processed one after the other:

$$\|x_1, x_2, \dots, x_m, \dots\| \text{ where } x_1 < x_2 < \dots < x_m < \dots$$

- Each task in the stream typically represents a discrete piece of work, such as a computation, data transformation, or process operation (e.g., data transfer or coordination), which contributes to the overall goal of the system or application
- In streaming computation, usually but not always, the stream length is **infinite** or very large
 - Terminology: stream **source** and **sink** are the first/last stages of the computation workflow (usually a **pipeline**) that produce and consume the stream of tasks, respectively
- The main parallel pattern dealing with stream parallelism is the **pipeline**
- Objective: **reduce the service-time**

Example: bikes assembly

- Work to assemble a single bike step by step:
 - 1) attach the frame; 2) Install the wheels; 3) fix the handlebars; 4) attach the pedals
 - $T_{bike} = 4 \cdot T_{task}$ (let's suppose that each phase takes about the same amount of time T_{task})
 - The cost for assembly N bikes with 1 Worker, is $T = N \cdot T_{bike}$
- If we consider an assembly line (**pipeline pattern**) of 4 Workers, we can model the N bikes to assemble as a **stream** of activities, and we consider the cost of «moving» elements from one stage to the next one as T_{move} , we have:
 - $T_c^{pipe}(N, 4) \cong (N + 4 - 1) \cdot (T_{task} + T_{move})$
 - We can **overlap** the cost of the different phases but we introduce an **overhead** for moving items
- What's the improvement (i.e., **speedup**), if any?
 - If N is large enough **and** $T_{move} \ll T_{task}$, the improvement is about 4X (i.e., $S(4) \approx 4$)

Task parallelism

- A task is a unit of work, i.e., **arbitrary sequential code**, that can be executed in parallel (using threads or processes) with other program's tasks
- In task parallelism, the computation is modeled as a **task graph**
 - Nodes are tasks, whereas edges are dependencies among tasks
 - The task graph is usually a DAG
- A task is ready to run **iff** all its predecessors in the graph have been completed, i.e., if all its input dependencies are ready
- Objective: **reduce the completion time**
- **Task parallelism may be used to express both data parallel and stream parallel computations**

Example: busy kitchen

- Busy kitchen with $k \geq 2$ chefs. The chefs prepare 2 meals: 1) spaghetti; 2) beef and vegetables.
 - Meal 1: cook spaghetti and assemble the plate (add tomatoes)
 - Meal 2: cook beef, cut vegetables and then assemble the plate
- Let's suppose that:
 - $T_{spaghetti} = T_{beef} = T_{vegetables} = T_{task}$ (each task takes about the same amount of time);
 - T_a is the time for assembling a plate, and this is also a task
 - All activities are modeled as a pool of different ready tasks, so that each chef picks the next ready task.
 - There is no single ordered chain shared by all plates
- Let $N = N_{m1} + N_{m2}$, the total work is about $W = T_{task} \cdot (N_{m1} + 2 \cdot N_{m2}) + T_a \cdot N$
- If the scheduling of tasks is greedy, then $T_c^{task}(N, k) \geq \max(\frac{W + \Delta(k)}{k}, T_{task} + T_a)$, where $T_{task} + T_a$ is the longest sequential path (called **span**), and $\Delta(k)$ is the **overhead related to task scheduling and synchronization among the k chefs**
- Under the assumption that the overhead of task scheduling and synchronization is small (i.e., $\Delta(k) \ll W$), and **for large enough N**

$$T_c^{task}(N, k) \approx \frac{W}{k}$$

- Under all these assumptions, the speedup is about k ($S(p) \approx \frac{W}{\frac{W}{k}} = k$)

Considerations

- **Parallelization introduces overhead**
 - Parallelism is not free; the overhead required to set up, coordinate, and integrate the parallel tasks must be minimized to achieve meaningful performance gains
- Parallelization requires **coordination** of activities and intermediate steps that introduce communication/synchronization (thus **overhead**)
- In most cases, a specific **Parallel Pattern** can be applied to solve a problem (e.g., map and pipeline)

Suggested Readings

- «**Structured Parallel Programming**» course notes by Prof. M. Danelutto (notes2024spm-Danelutto.pdf)